Technologies JS

Pr. Amal Ourdou
Email: amal.ourdou@gmail.com

What will you learn

- ✓ Advanced JavaScript
- ✓ Basic and Advanced understanding of how Node.js works
- ✓ Express JS
- ✓ Deep dive on TypeScript
- ✓ Angular

... and more

JavaScript

- What is JS
- History
- Basic JS
- Advanced JS

What is JS

JavaScript (JS) is a lightweight interpreted (or just-in-time compiled) programming language. While it is most well-known as the scripting language for Web pages, many non-browser environments also use it, such as Node.js. JavaScript is a prototype-based, multi-paradigm, single-threaded, dynamic language, supporting object-oriented styles.

History

- JavaScript was created in May 1995 in 10 days, by Brendan Eich. Eich worked at Netscape and implemented JavaScript for their web browser, Netscape Navigator.
- In 1996 JavaScript 1.0 was released
- In 1997 ECMAScript 1 was released & JavaScript became an ECMA standard (ECMA-262)
- In 1998 ECMAScript 2 was released & Netscape 42 was released with JavaScript 1.3
- 1999 ECMAScript 3 was released & Netscape 62 was released with JavaScript 1.5
- 2008 ECMAScript 4 was abandoned
- 2009 ECMAScript 5 was released
- 2011 IE 9 was the first browser to support ES5
- 2014 Full support for ES5 in all browsers
- 2015 ECMAScript 6 was released
- 2018 Full support for ES6 in all browsers*

*Internet Explorer 11 does not support ES6.

ECMAScript Language Specification

ECMA: European Computer Manufacturers Association

ECMA-262

- This is a standard published by Ecma International. It contains the specification for a general purpose scripting language.

ECMAScript

- ECMAScript or ES - is a general-purpose programming language, standardized by ECMA International according to the document ECMA-262.
- Although ECMAScript is inspired by JavaScript, they are different programming languages, and ECMAScript is not the specification of JavaScript.

JavaScript

- A general purpose scripting language where its standard is ECMAScript. .

By reading the ECMAScript specification, you learn how to create a scripting language. By reading the JavaScript documentation, you learn how to use a scripting language.

JavaScript

- JavaScript, alongside HTML and CSS is one of the core technologies of the world wide web
- JavaScript can be used to build server side applications
- JavaScript is also used in mobile app development to create cross platform apps that can run on both iOS and android
- JavaScript is also used to create desktop applications
- JavaScript is the most popular programming language according to the 2021 stack overflow developer survey

Evolving JavaScript: Don’t break the web

- One idea that occasionally comes up is to clean up JavaScript by removing old features and quirks. While the appeal of that idea is obvious, it has significant downsides.
- Let’s assume we create a new version of JavaScript that is not backward compatible and fix all of its flaws. As a result, we’d encounter some problems.

So what is the solution? The approach that was chosen for ES6 is called “One JavaScript”:

- New versions are always completely backward compatible (but there may occasionally be minor, hardly noticeable clean-ups).
- Old features aren’t removed or fixed. Instead, better versions of them are introduced. One example is declaring variables via let – which is an improved version of var.
- If aspects of the language are changed, it is done inside new syntactic constructs. That is, you opt in implicitly.

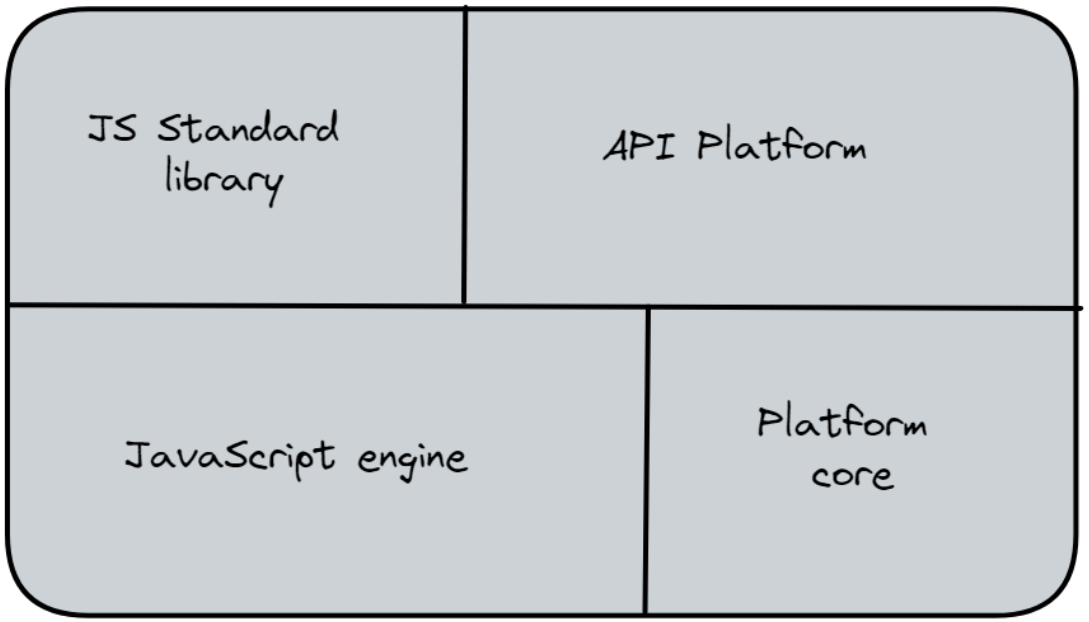
New features: https://github.com/tc39/proposals/blob/main/finished-proposals.md

Platforms

Platforms where JavaScript can be used:

- Web browser
- Node.js

The structure of browsers and Node.js



Structure of browsers and Node.js

The structures of the two JavaScript platforms web browser and Node.js are similar

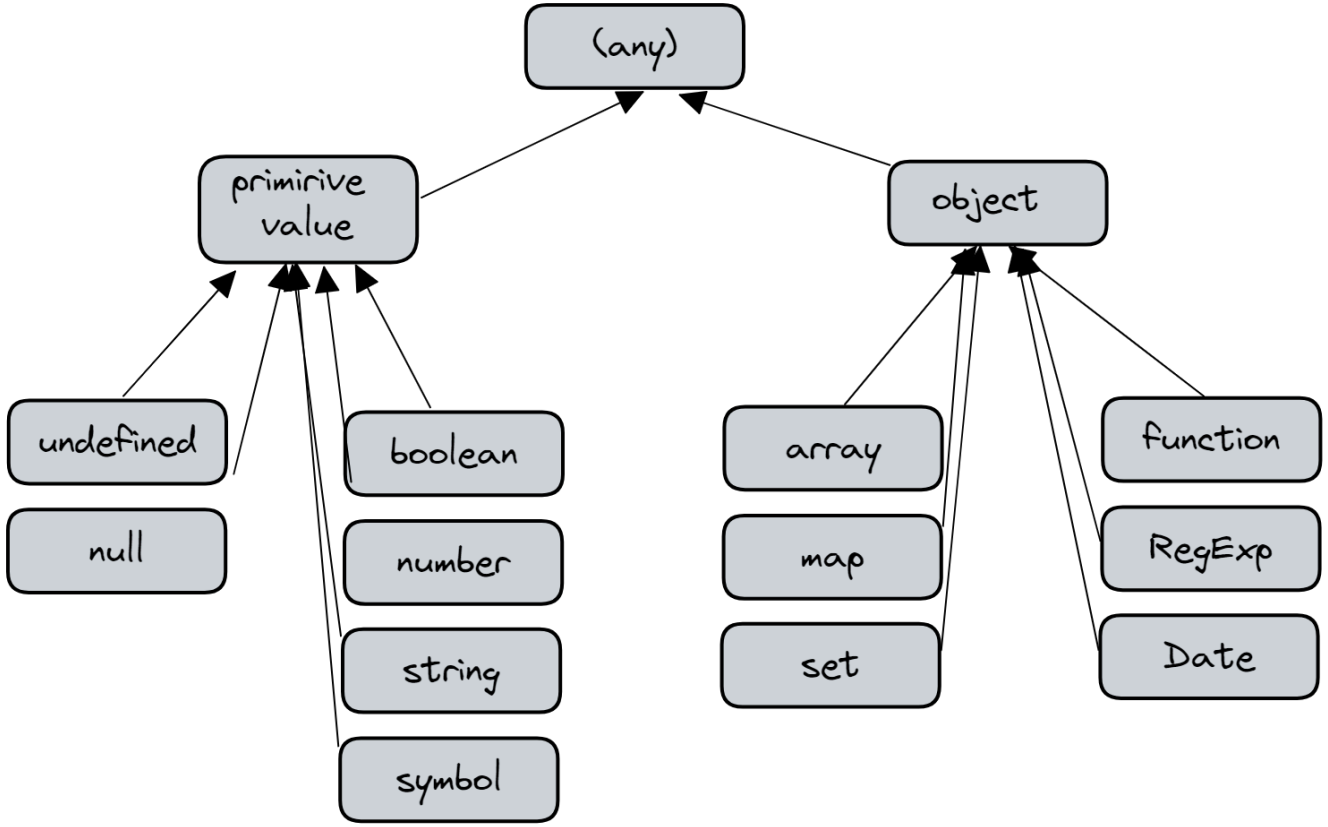
- The foundational layer consists of the JavaScript engine and platform-specific “core” functionality.
- Two APIs are hosted on top of this foundation:
  - The JavaScript standard library is part of JavaScript proper and runs on top of the engine.
  - The platform API are also available from JavaScript – it provides access to platform-specific functionality. For example:
    - In browsers, you need to use the platform-specific API if you want to do anything related to the user interface: react to mouse clicks, play sounds, etc.
    - In Node.js, the platform-specific API lets you read and write files, download data via HTTP, etc.

Basics

Comments

```
// single-line comment


/*
Comment with
multiple lines
*/
```

Values


(any)
primirive value
object
undefined
null
boolean
number
string
symbol
array
map
set
function
RegExp
Date

Primitive values:

- They are passed by value: when primitive values are assigned to variables or passed to functions, their contents are copied.
- They are compared by value: when comparing two primitive values, their contents are compared.

Objects: They are compound pieces of data.

- They are passed by identity: when objects are assigned to variables or passed to functions, their identities (think pointers) are copied.
- They are compared by identity: when comparing two objects, their identities are compared.
- The identity of an object is like a pointer (or a transparent reference) to the object's actual data on the heap (think shared main memory of a JavaScript engine).

Two common ways of creating objects:

Object literal starts and ends with curly braces {}.

```
const obj = {
  first: "Jane",
  last: "Doe",
};
```

Array literal starts and ends with square brackets [].

```
const fruits = ["strawberry", "apple"];
```

Objects:

The two operators typeof and instanceof let you determine what type a given value x has:

```
if (typeof x === 'string') ···
if (x instanceof Array) ···
```

> 💡 **Tip:** Rule of thumb: typeof is for primitive values; instanceof is for objects.

Logging to the console

```
// Printing a value to standard out (another method call)
console.log("Hello!");

// Printing error information to standard error
console.error("Something went wrong!");
```

Declaring variables

const (ES6 - 2015) creates immutable variable bindings: Each variable must be initialized immediately and we can’t assign a different value later.

You can NOT:

- Reassign a constant value,
- Reassign a constant array,
- Reassign a constant object,

But you CAN:

- Change the elements of constant array,
- Change the properties of constant object.

```
// Declaring and initializing x (immutable binding):
const x = 8;
// Would cause a TypeError:
// x = 9;
```

Declaring variables

let (ES6 - 2015) creates mutable variable bindings:

```
// Declaring y (mutable binding):
let y;
// We can assign a different value to y:
y = 3 * 5;
// Declaring and initializing z:
let z = 3 * 5;
```

Declaring variables

When to use let, const , var

1. Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers.

Declaring variables

All JavaScript variables must be identified with unique names. These unique names are called identifiers. Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume). The general rules for constructing names for variables (unique identifiers) are:

1. Names can contain letters, digits, underscores, and dollar signs.
2. Names must begin with a letter.
3. Names can also begin with $ and _ .
4. Names are case sensitive (y and Y are different variables).
5. Reserved words (like JavaScript keywords) cannot be used as names.

The scoop of a variable

The scope of a variable is the region of a program where it can be accessed.

- Block scoop

```
{
  let x = 2;
}
// x can NOT be used here
```

Variables declared with the ***var*** keyword can NOT have block scope.

```
{
  var x = 2;
}
// x CAN be used here
```

- Local / Function scoop

```
// code here can NOT use carName

function myFunction() {
  let carName = "Volvo";
  // code here CAN use carName
}

// code here can NOT use carName
```

- Global JavaScript Variables: A variable declared outside a function, becomes GLOBAL.

```
let carName = "Volvo";
// code here can use carName

function myFunction() {
  // code here can also use carName
}
```

Hoisting

Hoisting is JavaScript's default behavior of moving declarations to the top.

In JavaScript, a variable can be declared after it has been used.

```
x = 5; // Assign 5 to x
elem = document.getElementById("demo"); // Find an element
elem.innerHTML = x; // Display x in the element
var x; // Declare x
```

and

```
var x; // Declare x
x = 5; // Assign 5 to x
elem = document.getElementById("demo"); // Find an element
elem.innerHTML = x; // Display x in the element
```

have the same result

Hoisting

`let` and `const` declarations are hoisted to the top of their containing block scope. This means the JavaScript engine is aware of their existence within that scope before the code execution reaches their declaration.

However, unlike `var`, `let` and `const` variables are not initialized with `undefined` during the hoisting phase. They remain in an uninitialized state.

```
console.log(number);
// ReferenceError: Cannot access 'number' before initialization

console.log(number2);
// ReferenceError: number2 is not defined

let number = 10;
console.log(number);
```

Hoisting (Temporal Dead Zone)

The period from the beginning of the block scope until the `let` or `const` declaration is encountered and executed is known as the Temporal Dead Zone (TDZ).

Attempting to access a `let` or `const` variable within its TDZ will result in a `ReferenceError`. This error indicates that the variable exists but cannot be accessed yet because it has not been initialized.

> The `number` variable is in a temporal dead zone where JavaScript knows of its existence (because its declaration is hoisted) but it's not accessible (as it doesn't have an initialization).

Function declarations

- Simple function

```
// add1() has the parameters a and b
function add1(a, b) {
  return a + b;
}
// Calling function add1()
console.log(add1(5, 2));
```

A function declaration is always executed when entering its scope, regardless of where it is located within that scope. That enables you to call a function foo() before it is declared:

```
foo(); // OK
function foo() {
  return 123;
}
```

> Function declarations in JavaScript are hoisted to the top of their enclosing scope. This means that the JavaScript engine processes the declaration of the function before executing any other code within that scope.

Functions

Functions are values (anything you can do with a value, you can do with a function)

```
function greeting() {
  console.log("hello");
}
console.log(typeof greeting);
```

Output: ?

```
const fun = function greeting() {
  console.log("hello");
};
console.log(fun);
console.log(typeof fun);
fun();
// anonymous function (a function without a name)
const fun = function () {
  console.log("hello");
};
```

Output: ?

Functions

This

```
function greeting() {
  console.log("hello");
}
```

is a shortcut for

```
var greeting = function () {
  console.log("hello");
};
```

> Which is why the functions are hoisted!

Arrow function expressions

Arrow function expressions are used especially as arguments of function calls and method calls, If you declare a function via const or let, then it is not activated early. In the following example, you can only use `add2()` after its declaration.

```
// body is a code block
const add2 = (a, b) => {
  return a + b;
};
// Calling function add2(),
console.log(add2(5, 2));
// Equivalent to add2: body is an expression
const add3 = (a, b) => a + b;
```

Even if a function g() is not activated early, it can be called by a preceding function f() (in the same scope) if we adhere to the following rule: f() must be invoked after the declaration of g().

```
const f = () => g();
const g = () => 123;
// We call f() after g() was declared
```

Exercise

Rock Scissors Paper mini game

Code

Objects

```
// Creating a plain object via an object literal
const obj = {
  first: "Jane", // property
  last: "Doe", // property
  getFullName() {
    // property (method)
    return this.first + " " + this.last;
  },
};

// Getting a property value
console.log(obj.first); // Jane
// Setting a property value
obj.first = "Janey";

// Calling the method
console.log(obj.getFullName()); // 'Janey Doe'
// Delete a property
delete obj.first;
```

Arrays

An array is a list of values

```
const arr = ["a", "b", "c"];

// Getting an Array element
console.log(arr[1]);
// Setting an Array element
arr[1] = "R";
```

An array can have different types of values

```
const arr = ["a", 100, true, { name: "Amal" }];

// What's the output of these commands?
console.log(typeof arr);
console.log(Array.isArray(arr));
```

Arrays

Arrays are special objects hence have properties and methods

```
//Properties examples
arr.length; // Returns the number of elements


//Methodes examples
toString(); //converts an array to a string of (comma separated) array values
pop(); // removes the last element from an array and returns the value that was "popped out"
push(); // adds a new element to an array (at the end)
join(); // joins all array elements into a string, t behaves just like toString(), but in addition you can specify the separator
shift(); // removes the first array element and "shifts" all other elements to a lower index.
unshift(); // adds a new element to an array (at the beginning), and "unshifts" older elements
sort(); // Sorts the array
slice(); // Remove a value from an array
```

Maps

Before ES6, JavaScript didn't have a data structure for dictionaries. ES6 brought Maps, which are dictionaries from arbitrary values to arbitrary values. An instance of Map maps keys to values. A single key-value mapping is called an entry.

```
// Create a Map
const fruits = new Map([
  ["apples", 500],
  ["bananas", 300],
  ["oranges", 200],
]);
// Create a Map
const fruits = new Map();
// Set Map Values
fruits.set("apples", 500);
fruits.set("bananas", 300);
fruits.set("oranges", 200);

fruits.get("apples");
fruits.size;
fruits.delete("apples");
fruits.clear();
```

```javascript
// List all entries
let text = "";
for (const x of fruits.entries()) {
  text += x;
}

// List all keys
let text = "";
for (const x of fruits.keys()) {
  text += x;
}

// List all values
let text = "";
for (const x of fruits.values()) {
  text += x;
}
```

Sets

A JavaScript Set is a collection of unique values. Each value can only occur once in a Set. A Set can hold any value of any data type.

```
const letters = new Set(["a", "b", "c"]);


const letters = new Set();
// Add Values to the Set
letters.add("a");
letters.add("b");
letters.add("c");
for (const x of letters.values()) {
  console.log(x);
}
letters.forEach((x) => console.log(x));
// List all entries
let text = "";
letters.forEach(function (value) {
  text += value;
});
```

Conditional statement:

```
if (condition1) {
  //  block of code to be executed if condition1 is true
} else if (condition2) {
  //  block of code to be executed if the condition1 is false and condition2 is true
} else {
  //  block of code to be executed if the condition1 is false and condition2 is false
}
```

```
switch (expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
  // code block
}
```

Loops

```
for (let i = 0; i < cars.length; i++) {
  text += cars[i] + "<br>";
}
```

```
const person = { fname: "John", lname: "Doe", age: 25 };

let text = "";
for (let x in person) {
  text += person[x];
}
```

```
const numbers = [45, 4, 9, 16, 25];

let txt = "";
numbers.forEach(myFunction);

function myFunction(value, index, array) {
  txt += value;
}
```

```javascript
const cars = ["BMW", "Volvo", "Mini"];

let text = "";
for (let x of cars) {
  text += x;
}

let language = "JavaScript";

let text = "";
for (let x of language) {
  text += x;
}
```

```
while (i < 10) {
  text += "The number is " + i;
  i++;
}
```

```
do {
  text += "The number is " + i;
  i++;
} while (i < 10);
```

Exercise

Create a mini To do list app

Code

Classes

ES6 introduced classes. Classes can be seen as partitioning the single type object of the specification into subtypes – they give us more types than the limited 7 ones of the specification. Each class is the type of the objects that were created by it.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  describe() {
    return `Person named ${this.name}`;
  }
  static logNames(persons) {
    for (const person of persons) {
      console.log(person.name);
    }
  }
}
```

```
class Employee extends Person {
  constructor(name, title) {
    super(name);
    this.title = title;
  }
  describe() {
    return super.describe() + ` (${this.title})`;
  }
}

const jane = new Employee("Jane", "CTO");
console.log(jane.describe());
```

Exception handling

```
function throwsException() {
  throw new Error("Problem!");
}

function catchesException() {
  try {
    throwsException();
  } catch (err) {
    console.log(err.message);
  }
}
```

try-finally and ***try-catch-finally*** are also supported.

The code inside the finally clause is always executed at the end of a try statement – no matter what happens in the try block or the catch clause.

Advanced

Objects

Being able to create objects directly (without classes) is one of the highlights of JavaScript.

```
const myObject = {
  // object literal
  myProperty: 1,
  myMethod() {
    return 2;
  }, // comma!
  get myAccessor() {
    return this.myProperty;
  }, // comma!
  set myAccessor(value) {
    this.myProperty = value;
  }, // last comma is optional
};
```

Spreading into objects:

Spread syntax can be used when all elements from an object or array need to be included in a new array or object, or should be applied one-by-one in a function call's arguments list. There are three distinct places that accept the spread syntax:

- Function arguments list (myFunction(a, ...iterableObj, b))
- Array literals ([1, ...iterableObj, '4', 'five', 6])
- Object literals ({ ...obj, key: 'value' })

```
const original = {
  a: 1,
  b: {
    c: 3,
  },
};

const modifiedCopy = {
  // Spreading (...) copies one object “into” another one:
  ...original, // spreading
  d: 4,
};
```

Although the syntax looks the same, they come with slightly different semantics.

Only iterable values, like `Array` and `String`, can be spread in array literals and argument lists. Many objects are not iterable, including all plain objects that lack a Symbol.iterator method:

```
const obj = { key1: "value1" };
const array = [...obj]; // TypeError: obj is not iterable
```

On the other hand, spreading in object literals enumerates the own properties of the value. For typical arrays, all indices are enumerable own properties, so arrays can be spread into objects.

```
const array = [1, 2, 3];
const obj = { ...array }; // { 0: 1, 1: 2, 2: 3 }
```

All `primitives` can be spread in objects. Only strings have enumerable own properties, and spreading anything else doesn't create properties on the new object.

```
const obj = { ...true, ..."test", ...10 };
// { '0': 't', '1': 'e', '2': 's', '3': 't' }
```

We can use spreading to create a copy of an object original:

```
const original = { a: 1, b: { prop: true } };
const copy = { ...original };
```

The first level of copy is really a copy: If we change any properties at that level, it does not affect the original.
However, deeper levels are not copied. For example, the value of .b is shared between original and copy. Changing .b in the copy also changes it in the original.

> ❗ Deep copies of objects (where all levels are copied) are notoriously difficult to do generically. Therefore, JavaScript does not have a built-in operation for them (for now). If we need such an operation, we have to implement it ourselves.

Prototypes

Prototypes are JavaScript’s only inheritance mechanism: Each object has a prototype that is either null or an object. In the latter case, the object inherits all of the prototype’s properties.

In the browser's console create the following object

```
const myObject = {
  city: "Casablanca",
  greet() {
    console.log(`Greetings from ${this.city}`);
  },
};

myObject.greet(); // Greetings from Casablanca
```

What do you see when you type the object's name followed by a period into the console, like `myObject.`

What are these extra properties, and where do they come from?

Every object in JavaScript has a built-in property, which is called its **prototype**. The prototype is itself an object, so the prototype will have its own prototype, making what's called a prototype chain . The chain ends when we reach a prototype that has null for its own prototype.

When you try to access a property of an object: if the property can't be found in the object itself, the prototype is searched for the property. If the property still can't be found, then the prototype's prototype is searched, and so on until either the property is found, or the end of the chain is reached, in which case `undefined` is returned.

So when we call myObject.toString(), the browser:

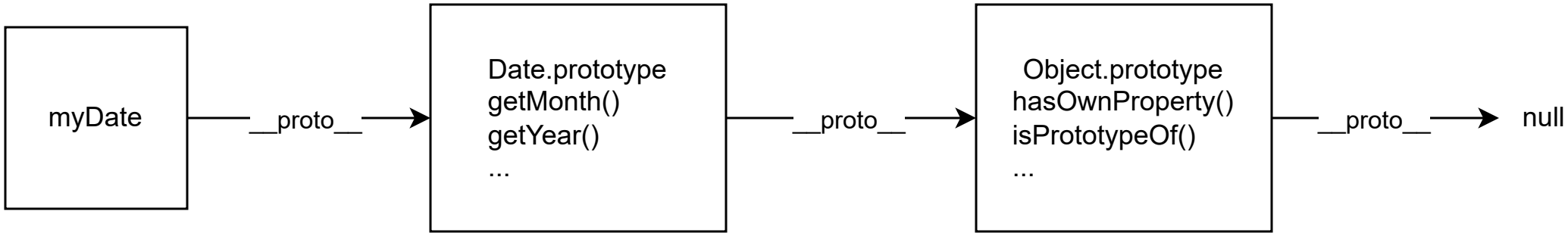
- looks for `toString` in `myObject`
- can't find it there, so looks in the prototype object of `myObject` for `toString`
- finds it there, and calls it.

The recommended way of getting the prototype of an object:

```
Object.getPrototypeOf(obj: Object) : Object
```

1. Now explain what's happening here:

```
const myDate = new Date();
let object = myDate;

do {
  object = Object.getPrototypeOf(object);
  console.log(object);
} while (object);
```



Viewer does not support full SVG 1.1

2. What happens if you define a property in an object, when a property with the same name is defined in the object's prototype?

There are various ways of setting an object's prototype in JavaScript, and here we'll describe two: Object.create() and constructors.

Using Object.create:

The `Object.create()` method creates a new object and allows you to specify an object that will be used as the new object's prototype.

```
const personPrototype = {
  greet() {
    console.log("hello!");
  },
};

const carl = Object.create(personPrototype);
carl.greet(); // hello!
```

Here we create an object `personPrototype`, which has a `greet()` method. We then use `Object.create()` to create a new object with `personPrototype` as its prototype. Now we can call `greet()` on the new object, and the prototype provides its implementation.

Using a constructor

In JavaScript, all functions have a property named `prototype`. When you call a function as a constructor, this property is set as the prototype of the newly constructed object (by convention, in the property named `__proto__`).

```
const personPrototype = {
  greet() {
    console.log(`hello, my name is ${this.name}!`);
  },
};

function Person(name) {
  this.name = name;
}

Object.assign(Person.prototype, personPrototype);
// or
// Person.prototype.greet = personPrototype.greet;
```

```
const mehdi = new Person("Mehdi");
mehdi.greet(); // hello, my name is Mehdi!
```

Properties that are defined directly in the object, like name here, are called own properties, and you can check whether a property is an own property using the static `Object.hasOwn()` method:

```
const oumaima = new Person("Oumaima");


console.log(Object.hasOwn(oumaima, "name")); // true
console.log(Object.hasOwn(oumaima, "greet")); // false
```

> ℹ️ Note: You can also use the non-static Object.hasOwnProperty() method here, but we recommend that you use Object.hasOwn() if you can.

Classes

Classes are basically a compact syntax for setting up prototype chains. But that is something we rarely see when working with them. Note that we don't need classes to create objects. We can also do so via object literals.

```
class Person {
  #firstName; // (A)
  constructor(firstName) {
    this.#firstName = firstName; // (B)
  }
  describe() {
    return `Person named ${this.#firstName}`;
  }
  static extractNames(persons) {
    return persons.map((person) => person.#firstName);
  }
}
const tarzan = new Person("Tarzan");
```

> [ES2022] private field .#firstName. In contrast to properties, private fields must be declared (line A) before they can be used (line B). A private field can only be accessed inside the class that declares it. It can't even be accessed by subclasses.

Subclass:

```
class Employee extends Person {
  constructor(firstName, title) {
    super(firstName);
    this.title = title; // (C)
  }
  describe() {
    return super.describe() + ` (${this.title})`;
  }
}

const jane = new Employee("Jane", "CTO");
```

Asynchronous programming

Asynchronous programming

In its most basic form, JS is synchronous, blocking, single-threaded language

- Synchronous
  - If we have two functions which log messages to the console, code executes top down, with only one line executing at any given time
- Blocking
  - No matter how long a previous process takes, the subsequent processes wont kick off until the former is completed
  - If function A had to execute an intensive chunk of code, JS has to finish that without moving on function B. Even if that code takes 10 seconds or 1 minutes
  - Web app runs in a browser and it executes an intensive an intensive chunk of code without returning control to the browser, the browser can appear to be frozen
- Single-threaded
  - A thread is simply a process that your JS program can use to run a task
  - Each thread can only do one task at a time
  - JS has just the one thread called the main thread for executing an code

Problem

- fetchDataFromDB(‘endpoint’) could take 1 second or even more
- During that time, we cant run any further code
- JS, if it simply proceeds to the next line without waiting, we have an error because data is not what we expect it to be
- We need a way to have asynchronous behaviour with JS

How ?

- Just JavaScript is not enough
- We need new pieces which are outside of JS to help us write asynchronous code which is where web browsers come into play
- Web browsers define functions and APIs that allow us to register functions that should not be executed synchronously, and should instead be invoked asynchronously when some kind of event occurs
- For example, that could be the passage of time (setTimeout or setInterval), the user's interaction with the mouse (addEventListener), or the arrival of data over the network (callbacks, Promises, async-await)

setTimeout()

- The setTimeout() function executes a particular block of code once after a specified time has elapsed
- The first parameter is a function to run, or a reference to a function defined elsewhere
- The second parameter is a number representing the duration in milliseconds to wait before executing code
- After the second parameter, you can pass in zero or more values that represent any parameters you want to pass to the function when it is run

```
function greet(name) {
  console.log(`Hello ${name}`);
}

setTimeout(greet, 2000, "Amal");
```

To clear a timeout, you can use the clearTimeout() method passing in the indentifier returned by setTimeout as a parameter

```
function greet(name) {
  console.log(`Hello`);
}

const timeoutId = setTimeout(greet, 2000);
clearTimeout(timeoutId);
```

setInterval()

- The setInterval() function repeatedly runs the same code over and over again at reuglar intervals
- The first parameter is a function to run, or a reference to a function defined elsewhere
- The second parameter is a number representing the duration in milliseconds
- After the second parameter, you can pass in zero or more values that represent any parameters you want to pass to the function when it is run

```
function greet(name) {
  console.log(`Hello`);
}

setInterval(greet, 2000);
```

- Intervals keep runing a task forever so you should clear the interval when appropriate

```
const intervalId = setInterval(greet, 2000);
clearInterval(intervalId);
```

- Timers and intervals are not part of JS itself. They are implemented by the browser and setTimeout and setInterval are basically names given to that functionality in JS
- Duration parameter is the minimum delay, not guaranteed delay
- It is possible to achieve the same effect as setInterval with a recursive setTimeout

Exercice: Write the equivalent of the effect below using setTimeout

```
setInterval(function run() {
  console.log("hello");
}, 1000);
```

- For setTimeout, the duration is guaranteed between executions. Irrespective of how long takes to run, the interval will remain the same
- For setInterval, the duration interval includes the time taken to execute the code you want to run
  - The code takes 40ms to run, the interval is 60ms
  - The code takes 50 ms to run, the interval is 50ms

> 💡 if the code can take longer to run then the time interval itself => Recursive setTimeout

Callbacks

- In JavaScript, functions are first class objects
- Just like an object, a function can be passed as an argument to a function
- A function can also be returned as values from other functions
- Any function that is passed as an argument to another function is called a callback function in JavaScript
- The function which accepts a function as an argument or returns a function is called a higher order function

```
function greet(name) {
  console.log(`Hello ${name}`);
}

function greetAmal(greetFn) {
  const name = "Amal";
  greetFn(name);
}

greetAmal(greet);
```

A callback which is executed immediatly is called a synchronous callback

Async Callbacks

- An asynchronous callback is a callback that is often used to continue or resume code execution after an asynchronous operation has completed
- Callbacks are used to delay the execution of a function until a particular time or event has occured
- Data fetching takes time and we can only run the function we want to after the data has been fetched and not immediately
- Callback functions allow you to delay the execution of a function

Problem

If you have multiple callback functions where each level depends on the result obtained from the previous level, the nesting of functions becomes so deep that the code becomes difficult to read and maintain

```
loadScript("1.js", function (error, script) {
  if (error) {
    handleError(error);
  } else {
    // ...
    loadScript("2.js", function (error, script) {
      if (error) {
        handleError(error);
      } else {
        // ...
        loadScript("3.js", function (error, script) {
          if (error) {
            handleError(error);
          } else {
            // ...continue after all scripts are loaded (*)
          }
        });
      }
    });
  }
```

Promises

- A promise is simply an object in JS
- A promise is always in one of the three states
  - Pending: which is initial state, neither fulfilled nor rejected
  - Fulfilled: meaning that the operation completed successfully
  - Rejected: meaning that the operation failed

Promises help us deal with asynchronous code in a far more simpler way compared to callbacks
Callback hell can be avoided with Promises

```javascript
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Resolve Test");
  }, 5000);
});

const promisee = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject("Reject test");
  }, 5000);
});

const onFulfillment = (result) => {
  console.log(result);
  console.log("On fulfillement");
};

const onReject = (error) => {
  console.log(error);
  console.log("On reject");
};

promise.then(onFulfillment);
promisee.catch(onReject);
```

Promise.all()

- Query multiple APIs and perform some actions but only after all the APIs have finished loading

```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "foo");
});

Promise.all([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});
```

- The `promise.all()` method takes an iterable of promises as an inputt ad returns a single promise that resolves to an array of the results of the input promises
- Returned promise will resolve when all the input's promises have resolved
- It rejects immediately if any of the input promises rejects or the promise throw an error, and will reject everything

Promise.allSettled()

`Promise.allSettled()` waits for all input promises to complete regardless of whether or not one of them is rejected

```
const promise1 = Promise.resolve(3);
const promise2 = 42;
const promise3 = new Promise((resolve, reject) => {
  setTimeout(reject, 100, "foo");
});

Promise.allSettled([promise1, promise2, promise3]).then((values) => {
  console.log(values);
});
```

Promise.race()

The `Promise.race()` method returns a promise that fulfills or rejects as soon as one of the input promises fulfills or rejects, with the value or the reason from that promise.

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, "one");
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, "two");
});

Promise.race([promise1, promise2]).then((values) => {
  console.log(values);
});
```

Exercise 1

```
/*
  1. Print out "Program Started" at the start of your code
  2. Create a Promise that resolve after 3 seconds
  3. Log out the promise while it's pending
  4. Print out "Program in progress..." as well
  5. Print out "Program complete" when the promise completes after 3 seconds
*/
```

Exercice 2

```
/*
  1. Print out "Program started" at the start of your code
  2. Create a Promise that resolves after 3 seconds and rejects after 2 seconds
  3. Log out the promise while it's pending
  4. Print out "Program in progress..." as well
  5. Print out "Program complete" if the promise fulfills
  6. Print out "Program failure" if the promise rejects
*/
```

Execrice 3

```
/*
  1. Print out "Program started" at the start of your code
  2. Create a Promise that resolves after 3 seconds
  3. Log out the promise while it's pending
  4. Print out "Program in progress..." as well
  5. Print out "Step 1 complete" when the first promise fulfills
  6. Have the first promise return another new Promise that will fulfill after 3 seconds with the message "Step 2 complete"


  7. Print out the message from the second promise after it fulfills ("step 2 complete")
*/
```

Async await

- Async await keyword was introduced in ES2017
- The async await keyword allow us to write completely synchronous-looking code while performing asynchronous tasks behind the scenes
- The async keyword is used to declare async functions
- Async functions are functions that are instance of the `AsyncFunction` constructor
- Unlike normal functions, async functions always return a promise which will be resolved with the value returned by the async function, or rejected with an exception uncaught within the async function

```
// Async function
async function greet() {
  return "Hello";
}
console.log(greet());

async function greet2() {
  return Promise.resolve("Hello");
}
greet2().then((value) => console.log(value));
```

await

- `await` keyword can be put infront of any async based function to pause your code until that promise settles and returns its results
- `await` only works inside async functions. Cannot use `await` in normal functions

```
async function greet() {
  let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("Hello"), 1000);
  });

  let result = await promise;

  console.log(result);
}
greet();
```

Sequential execution

```
function resolveHello() {
  return new Promise((resolve) => {
    setTimeout(function () {
      resolve("Hello");
    }, 2000);
  });
}

function resolveWorld() {
  return new Promise((resolve) => {
    setTimeout(function () {
      resolve("World");
    }, 1000);
  });
}

async function sequentialStart() {
  const hello = await resolveHello();
  console.log(hello); // Logs hello after 2 sec

  const world = await resolveWorld();
  console.log(world); // Logs world after 2+1 sec
}

sequentialStart();
```

Concurrent execution

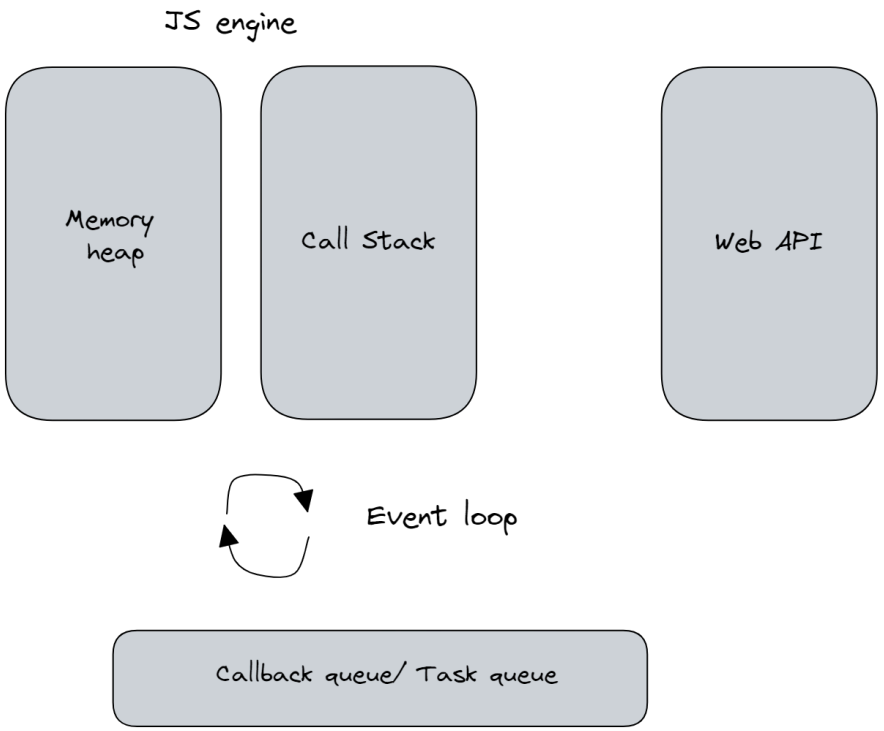
```
function resolveHello() {
  return new Promise((resolve) => {
    setTimeout(function () {
      resolve("Hello");
    }, 2000);
  });
}

function resolveWorld() {
  return new Promise((resolve) => {
    setTimeout(function () {
      resolve("World");
    }, 1000);
  });
}

async function concurrentStart() {
  const hello = resolveHello();
  const world = resolveWorld();

  console.log(await hello);
  console.log(await world);
}

concurrentStart();
```

Parallel execution

```
function resolveHello() {
  return new Promise((resolve) => {
    setTimeout(function () {
      resolve("Hello");
    }, 2000);
  });
}

function resolveWorld() {
  return new Promise((resolve) => {
    setTimeout(function () {
      resolve("World");
    }, 1000);
  });
}

function parallel() {
  Promise.all([
    (async () => console.log(await resolveHello()))(),
    (async () => console.log(await resolveWorld()))(),
  ]);
}

parallel();
```

Event Loop

JavaScript is single-threaded. Luckily, the browser gives us some features that the JavaScript engine itself doesn't provide: a Web API. This includes the DOM API, setTimeout, HTTP requests, and so on. This can help us create some async, non-blocking behavior.



Exemple: Synchronous code

```
console.log("First");

console.log("Second");

console.log("Third");
```

Exemple: Async setTimeput code

```
console.log("First");


setTimeout(() => {
  console.log("Second");
}, 2000);


console.log("third");
```

Exemple: Async Promise code

```
console.log("First");


const promise = fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then((response) => response.json())
  .then((json) => console.log(json));


console.log("second");
```

Exemple: setTimeout + Promise code

```
setTimeout(() => {
  console.log("First");
}, 0);

const promise = fetch("https://jsonplaceholder.typicode.com/posts/1")
  .then((response) => response.json())
  .then((json) => console.log(json));

let i;
for (i = 0; i < 100000000; i++) {
  //blocks for some seconds
}
console.log("second");
```

Here is an example that demonstrates that `setTimeout` does not run immediately after its timer expires:

```
const seconds = new Date().getTime() / 1000;

setTimeout(() => {
  // prints out "2", meaning that the callback is not called immediately after 500 milliseconds.
  console.log(`Ran after ${new Date().getTime() / 1000 - seconds} seconds`);
}, 500);

while (true) {
  if (new Date().getTime() / 1000 - seconds >= 2) {
    console.log("Good, looped for 2 seconds");
    break;
  }
}
```

Zero delays

Zero delay doesn't mean the call back will fire-off after zero milliseconds. Calling `setTimeout` with a delay of 0 (zero) milliseconds doesn't execute the callback function after the given interval.

```
console.log("this is the start");

setTimeout(() => {
  console.log("Callback 1: this is a msg from call back");
}); // has a default time value of 0

console.log("this is just a message");

setTimeout(() => {
  console.log("Callback 2: this is a msg from call back");
}, 0);

console.log("this is the end");
```